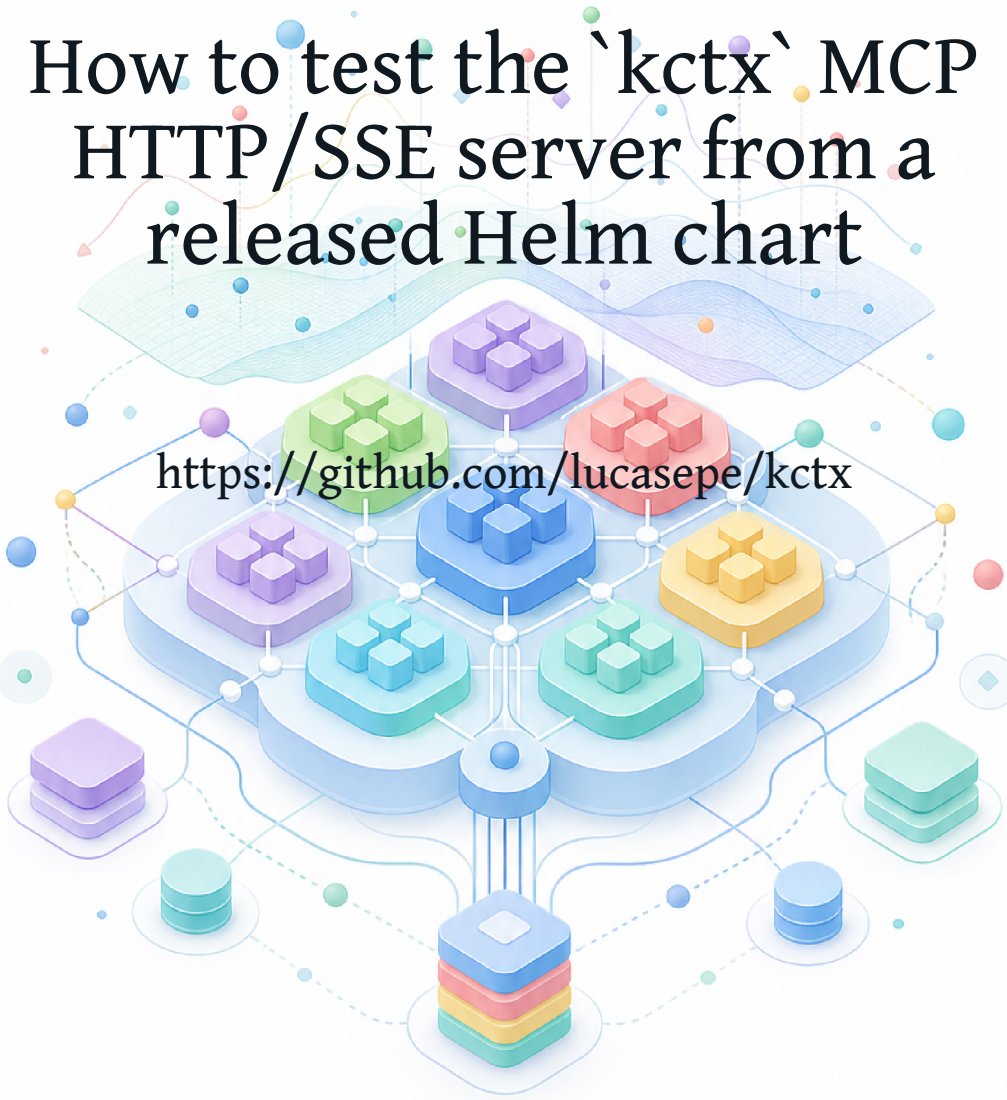
kctx MCP/SSE Release Test Guide

How to test the `kctx` MCP HTTP/SSE server from a released Helm chart

https://github.com/lucasepe/kctx



Index

Overview

This guide explains how to test the `kctx` MCP HTTP/SSE server from a released Helm chart.

The goal is to validate the complete release path:

- published chart package
- container image
- Helm values
- Kubernetes RBAC
- `kctx serve --mode mcp-sse`
- MCP/SSE transport
- real AI client configuration

The guide uses a local kind cluster and the Online Boutique demo workload from GoogleCloudPlatform/microservices-demo.
Online Boutique is a sample cloud-first application with multiple microservices, Kubernetes Services, and gRPC traffic. It gives the lab enough realistic Kubernetes objects for `kctx` to inspect without touching a production cluster.

No repository checkout is required. The commands use the released Helm chart, public Kubernetes manifests, and copy-paste shell snippets.

Current Scope

`kctx` exposes Kubernetes operational context through MCP tools:

```
get_namespace_health(namespace)
explain_resource(resource, namespace, name, eventLimit)
trace_service(namespace, name)
get_pod_graph(namespace, name)
dump_namespace(namespace)
```

The server is read-only. It does not mutate resources, restart Pods, apply manifests, or claim root cause.

Transport Status

`kctx` currently supports:

- `mcp`: local stdio MCP transport
- `mcp-sse`: HTTP/SSE MCP transport

The HTTP/SSE mode uses the older MCP HTTP+SSE shape:

```
GET  /mcp/sse
POST /mcp/message?sessionId=...
```

The MCP specification now recommends Streamable HTTP for new remote servers, while documenting HTTP+SSE as a backwards-compatibility path. `kctx` keeps HTTP/SSE because it is simple, testable, and still useful for current clients. Streamable HTTP is a possible future addition.

Reference:

- MCP transport specification: `https://modelcontextprotocol.io/specification/2025-06-18/basic/transports`

Security Boundary

This guide is for local labs, trusted internal networks, and community testing.

`kctx` does not yet include a built-in AuthN/AuthZ layer for the MCP HTTP/SSE server. Do not expose the endpoint publicly unless it is protected by an external gateway, tunnel policy, identity-aware proxy, VPN, or equivalent control.

Read-only does not mean harmless. The returned context may reveal namespaces, workload names, labels, topology, dependencies, Events, and failure modes.

Chapters

Prerequisites

Install the tools below before starting the lab.

Required

- Docker
- kind
- kubectl
- Helm
- curl
- Bash-compatible shell

Check them:

```
docker version
kind version
kubectl version --client
helm version
curl --version
bash --version
```

Optional

- ngrok, if you want a temporary HTTPS endpoint
- Codex, if you want to test the MCP server from Codex
- Claude Code, if you want to test the MCP server from Claude
- jq, if you want to inspect JSON responses manually

Check optional tools:

```
ngrok version
jq --version
```

No Repository Checkout Required

This guide intentionally avoids requiring a `kctx` repository checkout. Testers install

the released chart package directly and use inline shell snippets for the lab and smoke tests.

That keeps the feedback focused on what matters for release validation:

- the chart artifact
- the published container image
- the runtime Helm values
- the MCP/SSE endpoint behavior
- AI client compatibility

Version Variable

Set the release version you want to test:

```
export KCTX_VERSION=0.3.0
```

If `v0.3.0` has not been published yet, replace the value with the latest available release.

Ports

The examples use:

```
localhost:8888 -> kind NodePort 30088 -> kctx Service -> kctx Pod
```

If port `8888` is already in use, change the port mapping in your kind setup or use `kubectl port-forward` instead of NodePort.

Create A Local kind Lab

This chapter creates a reproducible Kubernetes lab with Online Boutique running in a `boutique` namespace.

Online Boutique comes from
GoogleCloudPlatform/microservices-demo.
It is a sample cloud-first application with multiple microservices showcasing Kubernetes, Istio, and gRPC. In this guide, it is used only as a realistic demo workload so the cluster has Services, Pods, EndpointSlices, owners, and Events for `kctx` to inspect.

Start kind

Create a kind cluster with a NodePort mapping from localhost port `8888` to Kubernetes NodePort `30088`:

```
cat <<'EOF' | kind create cluster --name kctx-lab --config -
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 30088
        hostPort: 8888
        protocol: TCP
EOF
```

Verify the cluster:

```
kubectl cluster-info
kubectl get nodes
```

Install Online Boutique

Deploy the demo workload:

```
kubectl create namespace boutique

kubectl apply \
  --namespace boutique \
  -f https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-demo/main/release/kubernetes-manifests.yaml
```

Verify the namespace:

```
kubectl get namespace boutique
kubectl get pods --namespace boutique
kubectl get svc --namespace boutique
```

Wait until most Pods are running:

```
kubectl wait pod \
  --namespace boutique \
  --for=condition=Ready \
  --all \
  --timeout=5m
```

Online Boutique is intentionally a real multi-service workload. It is not a `kctx` dependency; it is just a convenient way to populate the kind cluster with realistic microservice context.

Pick A Pod For Graph Testing

The `get_pod_graph` tool needs a concrete Pod name. Pod names are generated, so capture one dynamically:

```
export FRONTEND_POD="$(
  kubectl get pod \
    --namespace boutique \
    -l app=frontend \
    -o jsonpath='{.items[0].metadata.name}'
)"

echo "${FRONTEND_POD}"
```

Use this value later when testing `get_pod_graph`.

Install The Release Chart

This chapter installs `kctx` from the release Helm chart and enables MCP/SSE mode.

Install From GitHub Releases

Set the version:

```
export KCTX_VERSION="${KCTX_VERSION:-0.3.0}"
```

Install the chart package:

```
helm upgrade --install kctx \
  "https://github.com/lucasepe/kctx/releases/download/v${KCTX_VERSION}/kctx-${KCTX_VERSION}.tgz" \
  --namespace kctx-system \
  --create-namespace \
  --set env.mode=mcp-sse \
  --set env.requestTimeout=2m \
  --set env.kubeAPIBudget=1000 \
  --set service.type=NodePort \
  --set service.nodePort=30088
```

The important value is:

```
env:
  mode: mcp-sse
```

That maps to:

```
kctx serve --mode mcp-sse
```

Wait For Rollout

```
kubectl rollout status deployment/kctx --namespace kctx-system
kubectl get pods --namespace kctx-system
```

The Pod should become `Running`.

Verify Health Endpoints

```
curl http://localhost:8888/livez
curl http://localhost:8888/readyz
curl http://localhost:8888/version
```

Expected shape:

```
{"status":"ready"}
```

The MCP/SSE endpoint is:

```
http://localhost:8888/mcp/sse
```

Timeout And Budget

Two values matter when testing large namespaces:

```
env:
  requestTimeout: 2m
  kubeAPIBudget: 1000
```

`requestTimeout` bounds one MCP tool call. `kubeAPIBudget` limits how many guarded Kubernetes API operations one tool call can perform.

`dump_namespace` is the broadest tool and should be the first one to tune if a large namespace times out or exhausts the budget.

Port-Forward Alternative

If NodePort is not convenient, install without NodePort and forward the Service:

```
kubectl port-forward \
  --namespace kctx-system \
  svc/kctx \
  8888:8080
```

Keep that terminal open while testing.

Smoke Test MCP/SSE

This chapter verifies the MCP/SSE server before connecting an AI tool.

Standalone Smoke Script

The smoke test does not require a `kctx`
repository checkout. It downloads one small Bash script from GitHub and runs it
locally.

Run it:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

It opens the SSE stream, reads the session endpoint, sends `initialize`, lists tools, and calls:

```
get_namespace_health
trace_service
dump_namespace
```

By default it prints the JSON-RPC responses received from the SSE stream.

Include Pod Graph

If you captured a frontend Pod:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | env POD="${FRONTEND_POD}" \
    bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

If your shell does not preserve environment variables on the right side of a pipe, export the value first:

```
export POD="${FRONTEND_POD}"
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

That also calls:

```
get_pod_graph
```

Disable Namespace Dump

For very large namespaces, disable the broad dump during quick checks:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | env DUMP_NAMESPACE=false \
    bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

Manual curl Test

Open the SSE stream in one terminal:

```
curl -N http://localhost:8888/mcp/sse
```

The first event should look like:

```
event: endpoint
data: /mcp/message?sessionId=<session-id>
```

Copy the `data` path and use it in another terminal.

Initialize:

```
curl -X POST "http://localhost:8888/mcp/message?sessionId=<session-id>" \
  -H 'Content-Type: application/json' \
  -d '{
    "jsonrpc":"2.0",
    "id":1,
    "method":"initialize",
    "params":{
      "protocolVersion":"2025-06-18",
      "capabilities":{},
      "clientInfo":{"name":"manual","version":"dev"}
    }
  }'
```

List tools:

```
curl -X POST "http://localhost:8888/mcp/message?sessionId=<session-id>" \
  -H 'Content-Type: application/json' \
  -d '{
    "jsonrpc":"2.0",
    "id":2,
    "method":"tools/list",
    "params":{}
  }'
```

Call namespace health:

```
curl -X POST "http://localhost:8888/mcp/message?sessionId=<session-id>" \
  -H 'Content-Type: application/json' \
  -d '{
    "jsonrpc":"2.0",
    "id":3,
    "method":"tools/call",
    "params":{
      "name":"get_namespace_health",
      "arguments":{"namespace":"boutique"}
    }
  }'
```

Responses are delivered on the SSE stream, not in the POST response body. A successful POST returns:

```
202 Accepted
```

Expose The Endpoint With ngrok

Use this only for temporary testing.

`kctx` does not yet include built-in
AuthN/AuthZ for MCP/SSE. An ngrok URL is publicly reachable unless you protect it with ngrok access controls or another external policy. Do not use this for production access.

Start The Tunnel

Keep `kctx` reachable on localhost:

```
curl http://localhost:8888/readyz
```

Start ngrok:

```
ngrok http 8888
```

Copy the HTTPS forwarding URL, for example:

```
https://example.ngrok-free.app
```

Set:

```
export KCTX_MCP_BASE_URL="https://example.ngrok-free.app"
export KCTX_MCP_SSE_URL="${KCTX_MCP_BASE_URL}/mcp/sse"
```

Verify The Tunnel

```
curl "${KCTX_MCP_BASE_URL}/readyz"
curl "${KCTX_MCP_BASE_URL}/version"
```

Then test MCP/SSE:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- "${KCTX_MCP_SSE_URL}" boutique frontend
```

Practical Notes

- Free ngrok URLs can change between runs.
- Some AI tools require HTTPS for remote MCP servers.
- ChatGPT browser testing requires an HTTPS URL, so ngrok is useful for short-lived demos.
- Some AI tools may not support the older HTTP+SSE transport.
- If a client supports only Streamable HTTP, it will not work with the current `mcp-sse` endpoint.

For public demos, prefer a short-lived tunnel and a throwaway kind cluster.

Connect Codex

This chapter shows how to point Codex at the `kctx` MCP/SSE endpoint.

Codex MCP configuration and supported transports can vary by surface and version. The examples below use the `config.toml` shape used by the local Codex environment during development.

Local NodePort Endpoint

Use this when Codex runs on the same machine as kind:

```
[mcp_servers.kctx]
url = "http://localhost:8888/mcp/sse"
```

The usual config path is:

```
~/.codex/config.toml
```

After editing the file, start a new Codex session so the MCP server list is loaded again.

ngrok HTTPS Endpoint

If Codex needs an HTTPS endpoint:

```
[mcp_servers.kctx]
url = "https://example.ngrok-free.app/mcp/sse"
```

Replace the URL with your current ngrok forwarding URL.

Stdio Fallback

If your Codex surface does not support remote HTTP/SSE MCP servers, use the stdio mode from a machine that has Kubernetes access:

```
[mcp_servers.kctx]
command = "kctx"
args = ["serve", "--mode", "mcp"]
```

For a source checkout, if you are developing `kctx` locally:

```
[mcp_servers.kctx]
command = "go"
args = ["run", ".", "serve", "--mode", "mcp"]
```

The stdio server uses the local kubeconfig, so it is best for local development rather than in-cluster release-chart testing.

Prompts To Try

After starting a new Codex session, ask:

```
Use kctx to check the health of the boutique namespace.
```

Then:

```
Use kctx to trace the frontend Service in the boutique namespace.
```

Then:

```
Use kctx to dump the boutique namespace and summarize only factual signals.
```

For Pod graph testing:

```
Use kctx to get the graph for Pod <pod-name> in the boutique namespace.
```

Expected Tools

Codex should discover:

```
get_namespace_health
explain_resource
trace_service
get_pod_graph
dump_namespace
```

If Codex does not show or use these tools, verify the server with the standalone smoke script from Smoke Test MCP/SSE, then restart Codex and check the config path for the Codex surface you are using.

Connect Claude Code

This chapter shows how to connect Claude Code to the `kctx` MCP/SSE endpoint.

Claude Code documents multiple MCP transports, including remote HTTP and remote SSE. For new remote servers, Streamable HTTP is the preferred direction in the MCP specification. `kctx` currently exposes the older HTTP+SSE transport, so use Claude Code's SSE transport when testing this endpoint.

Reference:

- Claude Code MCP documentation: `https://docs.anthropic.com/en/docs/claude-code/mcp`
- MCP transport specification: `https://modelcontextprotocol.io/specification/2025-06-18/basic/transports`

Local NodePort Endpoint

Add the server:

```
claude mcp add --transport sse kctx http://localhost:8888/mcp/sse
```

Verify:

```
claude mcp list
claude mcp get kctx
```

Start or restart Claude Code in the project where you want to use the server.

ngrok HTTPS Endpoint

If Claude Code requires or benefits from HTTPS:

```
claude mcp add --transport sse kctx https://example.ngrok-free.app/mcp/sse
```

Replace the URL with your current ngrok forwarding URL.

JSON Configuration Alternative

Claude Code also supports JSON-based MCP configuration. A minimal remote SSE entry is:

```
{
  "mcpServers": {
    "kctx": {
      "type": "sse",
      "url": "http://localhost:8888/mcp/sse"
    }
  }
}
```

Use the scope that fits your workflow: local, project, or user.

Prompts To Try

```
Use kctx to check the health of the boutique namespace.
```

```
Use kctx to trace the frontend Service in the boutique namespace.
```

```
Use kctx to dump the boutique namespace. Only summarize factual signals and do not infer root cause.
```

```
Use kctx to get the Pod graph for <pod-name> in the boutique namespace.
```

Expected Behavior

Claude should discover the `kctx` tools and ask for permission according to your Claude Code settings.

If the server connects but tool calls fail, check:

- the kind cluster is still running
- Online Boutique still exists in the `boutique` namespace
- the ngrok URL has not changed
- the `kctx` Pod is running
- the endpoint works with the standalone smoke script from Smoke Test MCP/SSE

Remove The Server

```
claude mcp remove kctx
```

Troubleshooting

Use this chapter when the MCP/SSE endpoint or AI client does not behave as expected.

Pod Is Not Running

```
kubectl get pods --namespace kctx-system
kubectl describe pod --namespace kctx-system -l app.kubernetes.io/name=kctx
kubectl logs --namespace kctx-system deployment/kctx
```

If you see `CreateContainerConfigError`, inspect the chart values and security context:

```
helm get values kctx --namespace kctx-system
kubectl get deployment kctx --namespace kctx-system -o yaml
```

Health Endpoint Fails

```
curl -v http://localhost:8888/readyz
```

If this fails:

- verify kind is running
- verify the Helm release exists
- verify NodePort or port-forward is active
- check that `env.mode` is `mcp-sse`

```
helm get values kctx --namespace kctx-system
```

SSE Does Not Open

```
curl -N http://localhost:8888/mcp/sse
```

Expected first event:

```
event: endpoint
data: /mcp/message?sessionId=<session-id>
```

If you do not see this, check the `kctx` logs:

```
kubectl logs --namespace kctx-system deployment/kctx
```

POST Returns 202 But Nothing Appears

In MCP/SSE mode, POST requests acknowledge quickly with:

```
202 Accepted
```

The actual JSON-RPC response is sent later on the SSE stream. Keep the `curl -N` terminal open while posting messages from another terminal.

Namespace Dump Times Out

`dump_namespace` reads several Kubernetes resource families. Increase timeout and budget:

```
helm upgrade --install kctx \
  "https://github.com/lucasepe/kctx/releases/download/v${KCTX_VERSION}/kctx-${KCTX_VERSION}.tgz" \
  --namespace kctx-system \
  --set env.mode=mcp-sse \
  --set env.requestTimeout=5m \
  --set env.kubeAPIBudget=3000
```

For quick client checks, disable the dump in the standalone smoke script:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | env DUMP_NAMESPACE=false \
    bash -s --
```

AI Client Does Not See Tools

First verify the server without the AI client:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

Then check:

- the client supports the HTTP/SSE transport
- the MCP URL includes `/mcp/sse`

- the client was restarted after config changes
- the ngrok URL is current
- no corporate proxy is buffering or blocking SSE

Client Supports Only Streamable HTTP

The current `mcp-sse` mode is HTTP+SSE, not Streamable HTTP.

If a client only supports Streamable HTTP, it may fail even though the standalone smoke script works. In that case, use stdio mode if supported:

```
kctx serve --mode mcp
```

Streamable HTTP is a candidate future transport for `kctx`.

Cleanup

Remove the release:

```
helm uninstall kctx --namespace kctx-system
```

Delete the lab cluster:

```
kind delete cluster --name kctx-lab
```

Community Testing Checklist

Use this checklist when sharing `kctx` MCP/SSE with testers.

What To Ask Testers To Try

Ask testers to run the release-chart flow and report:

- operating system
- Kubernetes version
- kind version, if using kind
- `kctx` release version
- AI client name and version
- transport used: local NodePort, port-forward, ngrok, or internal URL
- whether the standalone smoke script passed
- whether the AI client discovered tools
- which tool calls succeeded or failed

Minimum Test Path

```
export KCTX_VERSION=0.3.0

cat <<'EOF' | kind create cluster --name kctx-lab --config -
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 30088
        hostPort: 8888
        protocol: TCP
EOF

kubectl create namespace boutique

kubectl apply \
  --namespace boutique \
  -f https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-demo/main/release/kubernetes-manifests.yaml

helm upgrade --install kctx \
  "https://github.com/lucasepe/kctx/releases/download/v${KCTX_VERSION}/kctx-${KCTX_VERSION}.tgz" \
  --namespace kctx-system \
  --create-namespace \
  --set env.mode=mcp-sse \
  --set env.requestTimeout=2m \
  --set env.kubeAPIBudget=1000 \
  --set service.type=NodePort \
  --set service.nodePort=30088

kubectl rollout status deployment/kctx --namespace kctx-system

curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- http://localhost:8888/mcp/sse boutique frontend
```

Useful AI Prompts

```
Use kctx to check the health of the boutique namespace. Only report factual signals.
```

```
Use kctx to trace the frontend Service in the boutique namespace.
```

```
Use kctx to dump the boutique namespace and summarize the highest-severity signals.
```

```
Use kctx to explain the frontend Pod in the boutique namespace.
```

What Good Feedback Looks Like

Good feedback includes concrete commands and outputs:

```
Client: Claude Code <version>
Transport: ngrok HTTPS, SSE
kctx: v0.3.0
Standalone smoke script: passed
AI client: discovered 5 tools
Failed call: dump_namespace
Observed error: timeout after 30s
Relevant logs: ...
```

Suggested Community Post

```
I am looking for testers for kctx v0.3.0 MCP/SSE support.

kctx is a read-only Kubernetes context engine. The new MCP/SSE mode lets AI
tools query structured Kubernetes facts such as namespace health, Service
traces, Pod graphs, and namespace dumps without giving the tool mutation
capabilities.

The test guide starts from a released Helm chart, uses kind + Google
Online Boutique as a realistic microservices demo, and includes optional Codex,
Claude Code, ChatGPT Developer Mode, and ngrok steps.

What I need most: feedback from real MCP clients, different OSes, and different Kubernetes setups.
```

Boundaries To Repeat Publicly

- `kctx` is read-only.
- It does not diagnose root cause.
- It does not remediate.
- MCP/SSE AuthN/AuthZ is not built in yet.
- Public exposure must be protected externally.

Connect ChatGPT

This chapter describes a lab-only way to test the `kctx` MCP/SSE endpoint from ChatGPT in the browser.

The flow is useful because it lets you ask ChatGPT questions about a local kind cluster where `kctx` is installed, without cloning this repository or running a local AI agent.

Important Safety Boundary

Use this only for local labs, demos, and short-lived testing.

The `kctx` MCP/SSE endpoint is read-only, but it does not yet include built-in AuthN/AuthZ. If you expose it through ngrok, the URL can be reached from the internet unless you protect it with ngrok access controls or another external policy.

Recommended testing boundary:

- use a disposable kind cluster
- use Online Boutique or another non-sensitive demo workload
- use a short-lived ngrok tunnel
- stop the tunnel immediately after testing
- do not expose a production cluster this way

Prerequisites

Complete these chapters first:

- Create A Local kind Lab
- Install The Release Chart
- Smoke Test MCP/SSE
- Expose The Endpoint With ngrok

You should have an HTTPS MCP/SSE URL:

```
https://example.ngrok-free.app/mcp/sse
```

Replace that example with your actual ngrok forwarding URL.

Enable Developer Mode

In ChatGPT:

1. Open your user menu.
2. Open **Settings**.
3. Open **Apps**.
4. Enable **Developer Mode**.

The exact labels may change over time. Look for the Apps settings area that allows adding a developer app, custom app, or MCP server URL.

Add The MCP/SSE Endpoint

Add a developer app using the ngrok MCP/SSE URL:

```
https://example.ngrok-free.app/mcp/sse
```

If ChatGPT asks for a name, use:

```
kctx
```

If the UI asks for a server URL, app URL, or MCP URL, use the full `/mcp/sse` URL, not only the ngrok domain.

Test From ChatGPT

Start with a narrow prompt:

```
Use kctx to check the health of the boutique namespace. Only report factual signals.
```

Then try:

```
Use kctx to trace the frontend Service in the boutique namespace.
```

For a broader context request:

```
Use kctx to dump the boutique namespace and summarize the highest-severity signals. Do not infer root cause.
```

For a Pod graph:

```
Use kctx to get the Pod graph for <pod-name> in the boutique namespace.
```

Expected Tools

ChatGPT should be able to discover the `kctx` MCP tools:

```
get_namespace_health
explain_resource
trace_service
get_pod_graph
dump_namespace
```

If the tools do not appear, verify the endpoint with the standalone smoke script first:

```
curl -fsSL https://raw.githubusercontent.com/lucasepe/kctx/main/scripts/mcp-sse-smoke.sh \
  | bash -s -- https://example.ngrok-free.app/mcp/sse boutique frontend
```

If ChatGPT Rejects The URL

This area is still moving quickly. If ChatGPT rejects the URL or does not call the tools:

- confirm Developer Mode is enabled
- confirm the URL starts with `https://`
- confirm the URL ends with `/mcp/sse`
- confirm the ngrok tunnel is still running
- confirm the smoke script works against the ngrok URL
- check whether your ChatGPT rollout expects a different remote MCP transport

`kctx` currently exposes HTTP+SSE for remote MCP testing. Streamable HTTP is a possible future transport.

Cleanup

When finished:

1. Remove the developer app from ChatGPT.
2. Disable Developer Mode if you do not need it.
3. Stop ngrok.
4. Uninstall `kctx` from the lab cluster.
5. Delete the kind cluster.

```
helm uninstall kctx --namespace kctx-system
kind delete cluster --name kctx-lab
```